[G R O V E W A R E]

The Unfair Advantage.

Project report: how a site selling nothing was built, hosted, wired up and made to feel like something was coming.

REACT 19 | VITE 8 | TAILWIND v4 | GITHUB PAGES | APPS SCRIPT | FIGMA MAKE | COPILOT

01

Purpose

GroveWare is a deliberately mysterious, fake "Counter-Strike cheat" website. It was built to:

LEARN THE TOOLS

Use Figma Make to create the base of the site, then migrate it to a real codebase and ship it with GitHub and Copilot, so each page can be found in Google search (Home, Evidence, Portal).

TEST PERSUASION

Make visitors feel something bigger is coming using clean animation, a polished UI and "evidence" of a product working, while never saying what is being sold.

THE CONCEPT

No product. No software. No hardware. Just an **idea posing as a product**. The copy is intentionally cryptic: *"The margin was not human."* · *"They asked how. We stopped answering."*

Contents

01 Purpose

02 Architecture and tech stack

03 Hosting: where and how

04 Domain and DNS

05 Request secure link: email → Sheet → auto-reply

06 Evidence page: video playback

07 Portal: the fake login

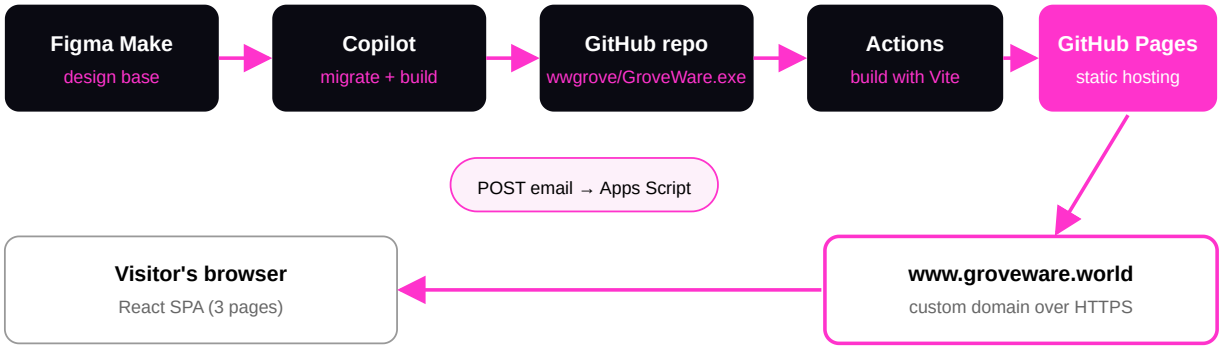
08 SEO and indexing

09 Takeaways

02

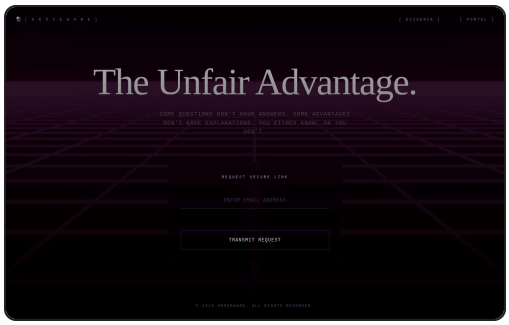
Architecture and tech stack

A fully static single-page app. There is no server of my own; the only "backend" is a Google Apps Script endpoint.

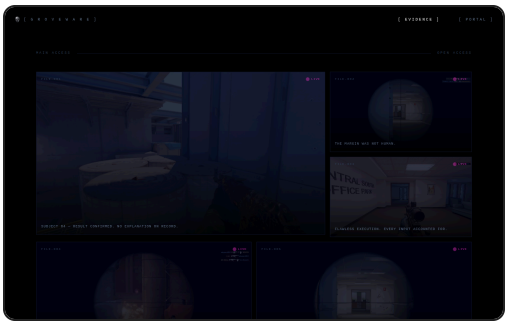


Layer	Technology	Role
UI	React 19 + TypeScript	Components and pages
Build	Vite 8	Bundling, dev server
Styling	Tailwind CSS v4	Utility classes, neon-on-black theme
Routing	react-router (`HashRouter`)	`/` , `/evidence` , `/portal`
Hosting / CI	GitHub Pages + Actions	Free hosting, auto deploy
Backend	Apps Script + Sheets + MailApp	Lead capture and auto-reply

The three pages



Home: tagline and "Request secure link"



Evidence: five "live" clips

03

Hosting: where and how

Where: GitHub Pages, served straight from the repository. **How:** a GitHub Actions workflow (`.github/workflows/deploy.yml`) builds and publishes on every push to `main` / `master`, or manually via `workflow_dispatch`.



Pipeline details

1. **Checkout** with Git LFS enabled, because the `.mp4` clips are large (`.gitattributes`).
2. **Install** pnpm 9 and Node 20, then run `pnpm install`.
3. **Build** with `pnpm vite build --base=./`. The relative base makes assets load correctly wherever the site is served.
4. **Upload** `dist/` with `actions/upload-pages-artifact`.
5. **Deploy** with `actions/deploy-pages`. Only one deployment runs at a time (`concurrency` group).

REQUIRED GITHUB SETTING

Settings → Pages → Source = **GitHub Actions** (not "Deploy from a branch").

.nojekyll

`public/.nojekyll` stops GitHub's Jekyll step from interfering with the built files.

WHY THIS MATTERS

Free hosting, HTTPS included, and every merged pull request goes live automatically, with no servers to maintain.

04

Domain and DNS

The custom domain is `www.groveware.world`. GitHub Pages knows which hostname to serve from a `CNAME` file, and then issues a free HTTPS certificate for it.



What was needed

1. In the repo: Settings → Pages → **Custom domain** = `www.groveware.world`. This writes the `CNAME` file.
2. At the registrar: add a **CNAME record**, host `www` → `wwgrove.github.io`.
3. *(Optional, for the bare domain)* add four **A records** to GitHub's Pages IPs so `groveware.world` redirects to `www`.
4. Wait for DNS propagation (minutes to a few hours) until GitHub's DNS check turns green.
5. Tick **Enforce HTTPS** once the certificate is issued.

Type	Host	Value
CNAME	`www`	`wwgrove.github.io`
A (optional)	`@`	`185.199.108.153`, `.109.153`, `.110.153`, `.111.153`

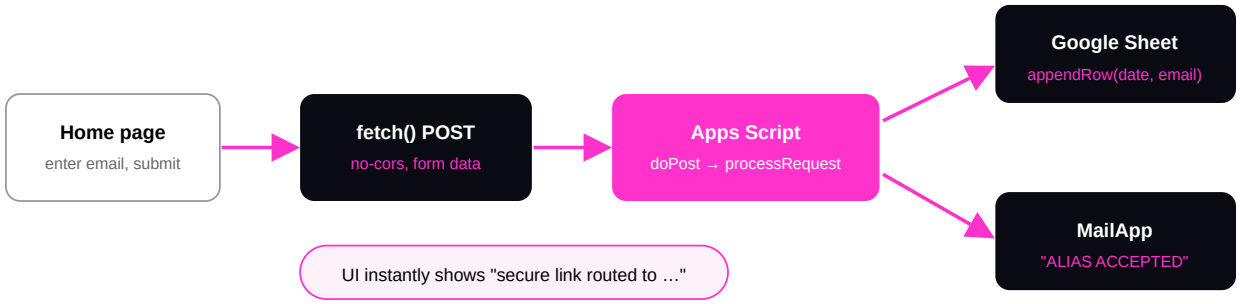
> **GOTCHA**
>
> The `CNAME` file must survive the build. Vite copies everything in `public/` into `dist/`, so keeping `public/CNAME` stops each Actions deploy from wiping the custom domain.

Note: the table shows the standard GitHub Pages setup. Adjust it to match the exact records you entered at your registrar.

05

Request secure link

The Home page takes an email address, logs it to a Google Sheet and sends an automated reply, all through one Google Apps Script Web App.



What the script does

1. Reads `e.parameter.email`; errors out if missing.
2. Appends `[timestamp, email]` to the first sheet (the lead list).
3. Sends the HTML email with a plain-text fallback via `MailApp.sendEmail`.
4. On failure, writes `ERROR: …` to the sheet so problems are visible.

`doGet` reuses the same handler, so it can be tested from a browser with `?email=`. `testSendEmail()` previews the email from the editor.

The automated email



DEPLOYMENT NOTES

Deploy as Web App with "Execute as: Me" and access "Anyone". The first run needs OAuth approval for Sheets and mail. Every script change needs a **new deployment version** for the live `/exec` URL to update.

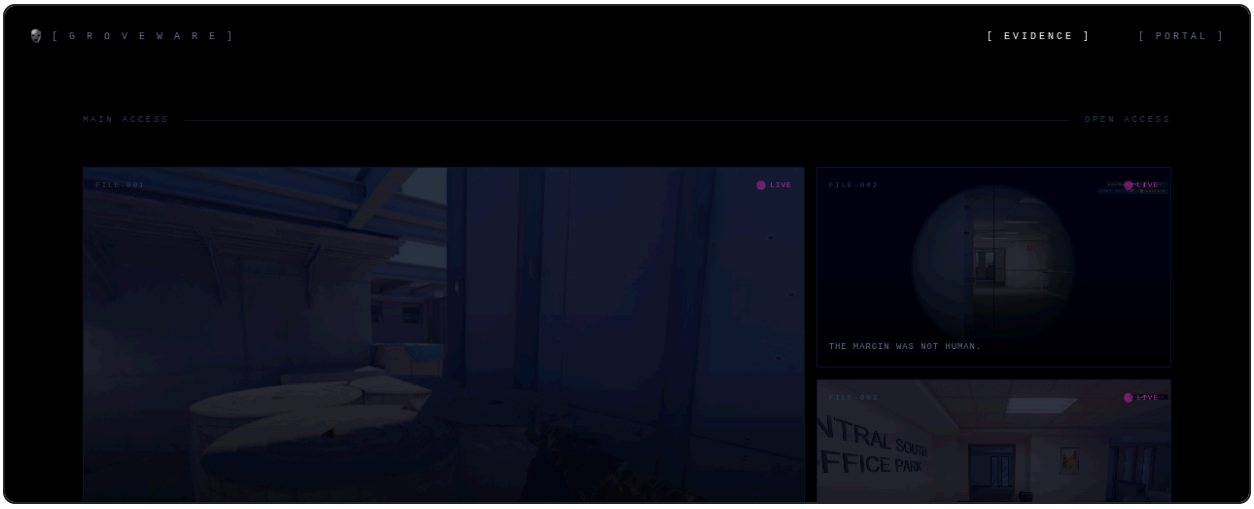
Limitations

- **Blind success:** with `no-cors` the browser cannot read the response, so the UI shows success even if the script failed.
- **Open endpoint:** the script URL is public in the bundle and anyone can submit any address. Add email validation, rate limiting and de-duplication.
- **Quotas:** `MailApp` has daily limits (about 100 recipients on free Gmail, about 1,500 on Workspace).
- **Theatre:** the "INITIALIZE LINK" button just opens the homepage, which suits the concept.

06

Evidence page: video playback

Five clips (`FILE-001` to `FILE-005`) sit in a bento grid: one large main clip, two stacked, then two wide. They loop as silent "live" footage and reveal themselves on hover.



Evidence page in its resting state: dimmed, muted, looping

AUTOPLAY, MUTED

`autoPlay loop muted playsInline` satisfies browser autoplay rules, so every clip loops on load.

HOVER = REVEAL

Mouse-enter unmutes, raises opacity from 20% to 100%, flips "Live" to "Audio On" and shows a custom cursor. Mouse-leave reverses it. Touch devices toggle on tap.

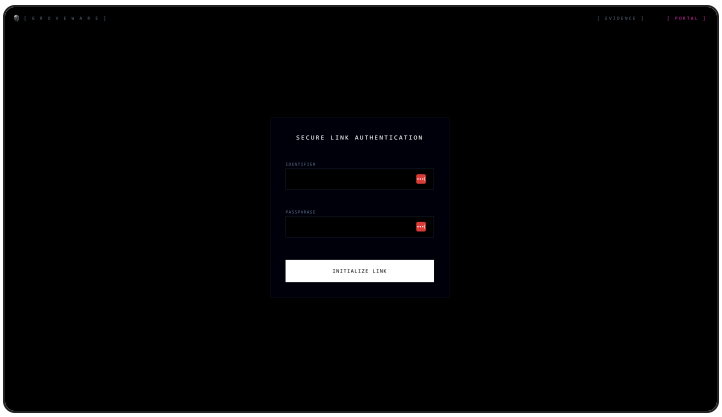
LOCKED FEED

Download, fullscreen, remote playback and picture-in-picture are disabled, and `pointer-events: none` removes player controls.

Finishing touches: a pinging pink dot, tiny mono-font file labels, a bottom gradient and cryptic captions give a "classified footage" mood. The clips are stored in Git LFS and bundled by Vite.

07

Portal: the fake login



/portal: "Secure Link Authentication"

`/portal` shows Identifier and Passphrase fields and an **Initialize Link** button.

It does nothing. There is no state, validation or network call. It exists only to suggest a members-only system and extend the illusion that a real product sits behind the site.

08

SEO and indexing

The goal was for Home, Evidence and Portal to appear in Google. `index.html` currently has one title ("GroveWare") and Open Graph tags (`og:title`, `og:description` = "The Unfair Advantage.", `og:image`).

HONEST CAVEAT

The app uses `HashRouter`, so URLs look like `groveware.world/#/evidence`. Google generally ignores everything after `#`, so sub-pages are unlikely to be indexed individually.

To index each page properly	How
Real URLs	Switch to `BrowserRouter` and add a `404.html` fallback (copy of `index.html`) for GitHub Pages, or pre-render per-page HTML
Per-page metadata	Unique `<title>` and meta description for each route
Discovery	Add `sitemap.xml` and verify the site in Google Search Console

09

Takeaways

FIGMA → CODE

Figma Make is great for fast visual scaffolding. A real repo plus Actions makes it shippable and indexable.

FREE, SOLID HOSTING

GitHub Pages plus a CNAME and one registrar record gives a reliable host with HTTPS.

SERVERLESS BACKEND

Apps Script is a quick way to get a Sheet and an automated email behind a form, with no infrastructure.

ATMOSPHERE SELLS

Motion, restraint and withholding information can carry a concept with no product behind it.

BOTTOM LINE

An idea, presented well enough, can feel like a product. Built to learn Figma Make and Copilot, and it worked.